```
#endif
        .mmap =     sock_mmap,
        .open =     sock_no_open, /* special open code to disallow open via /
proc */
        .release =  sock_close,
        .fasync =   sock_fasync,
        .sendpage = sock_sendpage,
        .splice_write = generic_splice_sendpage,
        .splice_read =   sock_splice_read,
};
```

In order to support the movement of the socket address from the kernel space to user space (elucidated later in this column), we need to define the following:

#define MAX_SOCK_ADDR 128

It specifies the length of MAX_SOCK_ADDR. MAX_SOCK_ADDR should be defined as large value in order to handle the lengthiest struct. And this is the sole purpose of MAX_SOCK_ADDR. You may note that it is 108 for the UNIX domain, 16 for IP (and IPX too), 24 for IPv6 and about 80 for AX.25. This should be larger than the AF_UNIX size, which is given as:

```
static int unix_mkname(struct sockaddr_un *sunaddr, int len, unsigned
*hashp)
{
        if (len <= sizeof(short) || len > sizeof(*sunaddr))
                return -EINVAL;
        if (!sunaddr || sunaddr->sun_family != AF_UNIX)
                return -EINVAL;
        if (sunaddr->sun_path[0]) {
                /*
                 * This may look like an off by one error but it is a bit more
                 * subtle. 108 is the longest valid AF_UNIX path for a binding.
                 * sun_path[108] doesnt as such exist. However in kernel space
                 * we are guaranteed that it is a valid memory location in our
                 * kernel address buffer.
                 */
                ((char *)sunaddr)[len] = 0;
                len = strlen(sunaddr->sun_path)+1+sizeof(short);
                return len;
        }

        *hashp = unix_hash_fold(csum_partial(sunaddr, len, 0));
        return len;
}
```

The whole process should also help us in locating the messy bits by slicing the large chunk. I am referring to all the steps involved in the movement of the socket address. And *unix_mkname()* is a critical one. One should understand that when we pass *len*, it is taken directly to the system call, but *sunaddr* is already there in the kernel space. Hence we need to check *sizeof(sockadd_un)* and these are handled by the kernel in order to check if the values are corrupted (messy). And one of the methods of doing this is by slicing the whole part into small pieces.

We have the *move_addr_to_kernel()* function, which will copy the given socket address to kernel-space.

```
int move_addr_to_kernel(void __user *uaddr, int ulen, struct sockaddr
*kaddr)
{
        if (ulen < 0 || ulen > sizeof(struct sockaddr_storage))
                return -EINVAL;
        if (ulen == 0)
                return 0;
        if (copy_from_user(kaddr, uaddr, ulen))
                return -EFAULT;
        return audit_sockaddr(ulen, kaddr);
}
```

In the code given above, *uaddr* represents an address in user-space, and *kaddr* an address located in kernel-space; *ulen* is the length corresponding to user-space. The code is easy to comprehend; it simply copies the address to the kernel space. If the address is too long, then the *EINVAL* error code is returned, and if the address is invalid, then *EFAULT* is returned. Otherwise, '0' is returned.

The corresponding function that copies an address to user-space is *move_addr_to_user()*.

```
int move_addr_to_user(struct sockaddr *kaddr, int klen, void __user *uaddr,
                      int __user *ulen)
{
        int err;
        int len;

        err = get_user(len, ulen);
        if (err)
                return err;
        if (len > klen)
                len = klen;
        if (len < 0 || len > sizeof(struct sockaddr_storage))
                return -EINVAL;
        if (len) {
                if (audit_sockaddr(klen, kaddr))
                        return -ENOMEM;
                if (copy_to_user(uaddr, kaddr, len))
                        return -EFAULT;
        }
        /*
         *      "fromlen shall refer to the value before truncation.."
         *                      1003.1g
         */
        return __put_user(klen, ulen);
}
```

fromlen is an integer variable obtained when we use *sizeof(from)*, where we have *struct sockaddr from*. Here, too,